

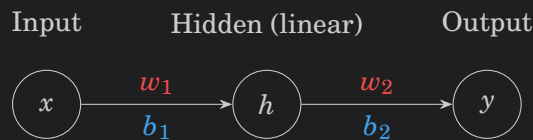
Neural Networks & Deep Learning

Bhuv

September 25, 2025

Linear Neural Network. (Simple)

We never use Linear Neural Networks (1 input, 1 hidden layer, 1 output). Usually we use multiple inputs and multiples hidden layers. This is just for understanding. So, let's consider a simple example.



Forward Propagation.

So here, after the input, we have a linear transformation to the hidden layer.

$$O_x = w_1 \cdot x + b_1$$

This is the output after the linear transformation. We have to apply the activation function to introduce non-linearity to the learning.

$$h = f_a(O_x) = f_a(w_1 \cdot x + b_1)$$

And now the hidden layer undergoes the same process as the input layer.

$$O_h = w_2 \cdot h + b_2$$

And finally, we apply the activation function again to get the output.

$$y = f_a(O_h) = f_a(w_2 \cdot h + b_2)$$



If we don't apply the activation function, the model will only learn linear relationships, limiting its capacity to capture complex patterns in the data. And also the activation function helps in normalizing the output, making it easier to handle and interpret. AND, the activation function is the same for every layer, we use only a single activation function for the entire network.

We got the output, now what ? We check whether it is correct or not and then punish the model if it is wrong (which it will be, in the beginning stages of learning). How do we do that ??

We calculate the loss, which is a measure of how far the model's predictions are from the actual target values. The loss function quantifies this difference, and our goal is to minimize it during training.

Loss Function & Gradient Descent.

The most basic and wide-known loss function is the MSE Loss, which is just the mean squared error. Continuing from the previous equations, let's define the correct answer (label) as y_{true} and our predicted output as y_{pred}

$$\Delta = \frac{1}{n} \sum_{i=1}^n \left(y_{\text{pred}}^i - y_{\text{true}}^i \right)^2$$

Since we have only one output,

$$\Delta = \left(y_{\text{pred}} - y_{\text{true}} \right)^2$$

This is the Loss function, which we want to minimize to minimize the error in predictions. To minimize the loss, we use a technique called Gradient Descent. The idea is to compute the gradient (or derivative) of the loss function with respect to the model parameters (weights and biases) and then update the parameters in the opposite direction of the gradient.

The update rule for a parameter θ is given by:

$$\theta = \theta - \eta \cdot \frac{\partial \Delta}{\partial \theta}$$

where η is the learning rate, a hyperparameter that controls how much we adjust the parameters at each step.

Backpropagation.

How it works is that first we calculate the "gradient" (slope) of the function with respect to each weight and minimize the error w.r.t to that particular weight by changing the value of the weight towards the downhill of the slope, ultimately leading to a minimum. This happens for every single weight and bias in the network. For that, we need to find the gradient w.r.t to that particular parameter. We use chain rule to achieve that.

$$\frac{\partial \Delta}{\partial w_2} = \frac{\partial \Delta}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial O_h} \cdot \frac{\partial O_h}{\partial w_2}$$

Now, calculating the direct equations for the above gradient,

$$\frac{\partial \Delta}{\partial w_2} = \frac{\partial \Delta}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial O_h} \cdot \frac{\partial O_h}{\partial w_2} = 2(y_{\text{pred}} - y_{\text{true}}) \cdot f'_a(O_h) \cdot h$$

Sigmoid Activation function.

Here, let's take the Sigmoid activation function, which is defined as

$$f_a(x) = \frac{1}{1 + e^{-x}}$$

The derivative of the Sigmoid function is given by

$$f'_a(x) = f_a(x) \cdot (1 - f_a(x))$$

So, we get the final value as,

$$\frac{\partial \Delta}{\partial w_2} = 2(y_{\text{pred}} - y_{\text{true}}) \cdot \frac{1}{1 + e^{O_h}} \cdot \left(\frac{e^{-O_h}}{1 + e^{-O_h}} \right) \cdot h$$

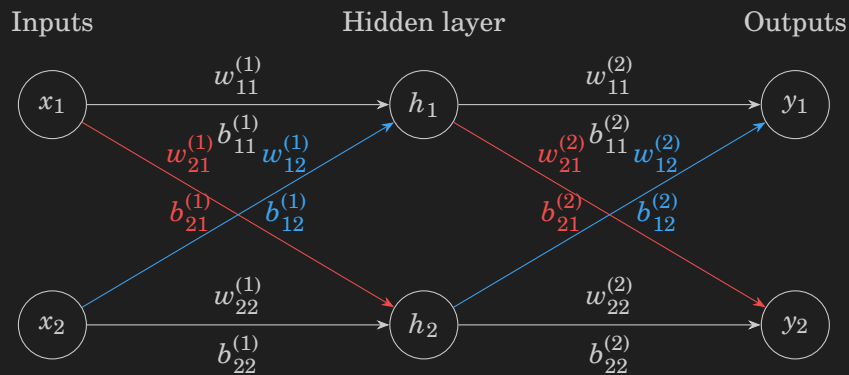
Now for the other weight, it'll be

$$\frac{\partial \Delta}{\partial w_1} = \frac{\partial \Delta}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial O_h} \cdot \frac{\partial O_h}{\partial h} \cdot \frac{\partial h}{\partial O_x} \cdot \frac{\partial O_x}{\partial w_1}$$

You expand this and find the value of the gradient and apply gradient descent for the respective weights and change them. The same thing happens for biases.

2-Dimensional Neural Network.

In a 2-dimensional neural network, we have 2 inputs, 1 hidden layer (2 units) and 2 outputs. The architecture looks something like.



Forward Propagation.

Here, the values of inputs are passed by multiplying with their weights and biases towards the neuron its going through next. For example, from x_1 to h_1 , the output for x_1 will be,

$$O_{x_1 \rightarrow h_1} = w_{11}^{(1)} \cdot x_1 + b_{11}^{(1)}$$

But h_1 also gets weighted value from x_2 too. So, to simplify things, I'll be directly writing the final input for h_1 , including the activated function.

$$h_1 = \mathbf{A} \left(w_{11}^{(1)} \cdot x_1 + b_{11}^{(1)} + w_{12}^{(1)} \cdot x_2 + b_{12}^{(1)} \right)$$

Now, same thing happens for h_2 too. So let's move on to the propagation from hidden layer to output layer.

$$y_1 = \mathbf{A} \left(\underbrace{w_{11}^{(2)} \cdot h_1 + b_{11}^{(2)} + w_{12}^{(2)} \cdot h_2 + b_{12}^{(2)}}_{O_{y1}} \right)$$

$$y_2 = \mathbf{A} \left(\underbrace{w_{21}^{(2)} \cdot h_1 + b_{21}^{(2)} + w_{22}^{(2)} \cdot h_2 + b_{22}^{(2)}}_{O_{y2}} \right)$$

Gradient Descent & Backpropagation.

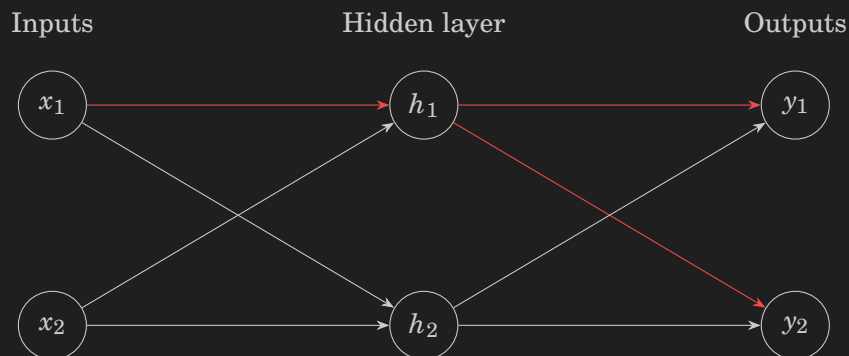
First, we calculate the loss function. We are gonna use MSE here. So, the equation is gonna be like

$$\Delta = \frac{1}{2} \sum_{i=1}^2 (y_i - \hat{y}_i)^2$$

You realise that despite summing them up, the gradients for each output are calculated independently during backpropagation because of partial derivatives. So, first we find the gradient of the loss function w.r.t $w_{11}^{(2)}$.

$$\frac{\partial \Delta}{\partial w_{11}^{(2)}} = \frac{\partial \Delta}{\partial y_1} \cdot \frac{\partial y_1}{\partial O_{y1}} \cdot \frac{\partial O_{y1}}{\partial w_{11}^{(2)}}$$

It's very similar for the rest of the weights and biases that are from the hidden layer to the output layer. Things change when we want to find the gradient for the weights and biases from the input layer to the hidden layer. For calculating the gradient of the loss function w.r.t $w_{11}^{(1)}$, we have to backpropagate via the two paths that are possible from the output layer to the first input.



So, the equation for the gradient w.r.t $w_{11}^{(1)}$ is

$$\frac{\partial \Delta}{\partial w_{11}^{(1)}} = \frac{\partial \Delta}{\partial y_1} \cdot \frac{\partial y_1}{\partial O_{y1}} \cdot \frac{\partial O_{y1}}{\partial h_1} \cdot \frac{\partial h_1}{\partial O_{h1}} \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}} + \frac{\partial \Delta}{\partial y_2} \cdot \frac{\partial y_2}{\partial O_{y2}} \cdot \frac{\partial O_{y2}}{\partial h_1} \cdot \frac{\partial h_1}{\partial O_{h1}} \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}}$$

And its similar for $w_{12}^{(1)}$.

First Step Towards Generalization.

We'll first generalize this for 2-D by making use of matrices and vectors. Then we'll generalize it for n-dimensions. Now, we'll introduce a new term $\delta_i^{(L)}$ where i is the index of the neuron in layer L .

$$\delta_1^{(3)} = \frac{\partial \Delta}{\partial y_1} \cdot \frac{\partial y_1}{\partial O_{y1}}$$

This is actually called the loss term of the neuron. This is quite useful in generalization. So basically, we can write

$$\frac{\partial \Delta}{\partial w_{11}^{(2)}} = \delta_1^{(3)} \cdot \frac{\partial O_{y1}}{\partial w_{11}^{(2)}} = \delta_1^{(3)} \cdot h_1$$

Now, we have

$$\delta^{(3)} = \begin{bmatrix} \frac{\partial \Delta}{\partial y_1} \cdot \frac{\partial y_1}{\partial O_{y1}} \\ \frac{\partial \Delta}{\partial y_2} \cdot \frac{\partial y_2}{\partial O_{y2}} \end{bmatrix} = \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \end{bmatrix}$$

This is basically a vector of loss terms of the output layer. Now, we have the vector of outputs of the previous (hidden) layer as

$$h = [h_1 \quad h_2]$$

Now, here's where the magic of matrices happens. The gradient of the weights from hidden layer to output layer can be represented as the multiplication of the loss term vector and the output vector of the previous layer.

$$\frac{\partial \Delta}{\partial w_{ij}^{(2)}} = \begin{bmatrix} \delta_1^{(3)} \\ \delta_2^{(3)} \end{bmatrix} \cdot [h_1 \quad h_2] = \begin{bmatrix} \delta_1^{(3)} \cdot h_1 & \delta_1^{(3)} \cdot h_2 \\ \delta_2^{(3)} \cdot h_1 & \delta_2^{(3)} \cdot h_2 \end{bmatrix}$$

Now, we do the same for the weights from input layer to hidden layer. First, we find the loss term vector for the hidden layer. It can be defined in general terms as,

$$\delta_j^{(l)} = \mathbf{A}'(O_{h_j}) \cdot \sum_{i=1}^l \delta_i^{(l+1)} \cdot w_{ij}^{(l)}$$

Don't worry if you don't understand this, it will be clear in a bit. So, for our case, we have our gradient as,

$$\frac{\partial \Delta}{\partial w_{11}^{(1)}} = \frac{\partial \Delta}{\partial y_1} \cdot \frac{\partial y_1}{\partial O_{y1}} \cdot \frac{\partial O_{y1}}{\partial h_1} \cdot \frac{\partial h_1}{\partial O_{h1}} \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}} + \frac{\partial \Delta}{\partial y_2} \cdot \frac{\partial y_2}{\partial O_{y2}} \cdot \frac{\partial O_{y2}}{\partial h_1} \cdot \frac{\partial h_1}{\partial O_{h1}} \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}}$$

We are gonna simplify this.

$$\begin{aligned} \frac{\partial \Delta}{\partial w_{11}^{(1)}} &= \delta_1^{(3)} \cdot w_{11}^{(2)} \cdot \mathbf{A}'(O_{h1}) \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}} + \delta_2^{(3)} \cdot w_{21}^{(2)} \cdot \mathbf{A}'(O_{h1}) \cdot \frac{\partial O_{h1}}{\partial w_{11}^{(1)}} \\ \frac{\partial \Delta}{\partial w_{11}^{(1)}} &= \frac{\partial O_{h1}}{\partial w_{11}^{(1)}} \cdot \left(\underbrace{\delta_1^{(3)} \cdot w_{11}^{(2)} \cdot \mathbf{A}'(O_{h1}) + \delta_2^{(3)} \cdot w_{21}^{(2)} \cdot \mathbf{A}'(O_{h1})}_{\delta_1^{(2)}} \right) \end{aligned}$$

Now, if you go back and compare this with the definition, it matches perfectly. So, we have

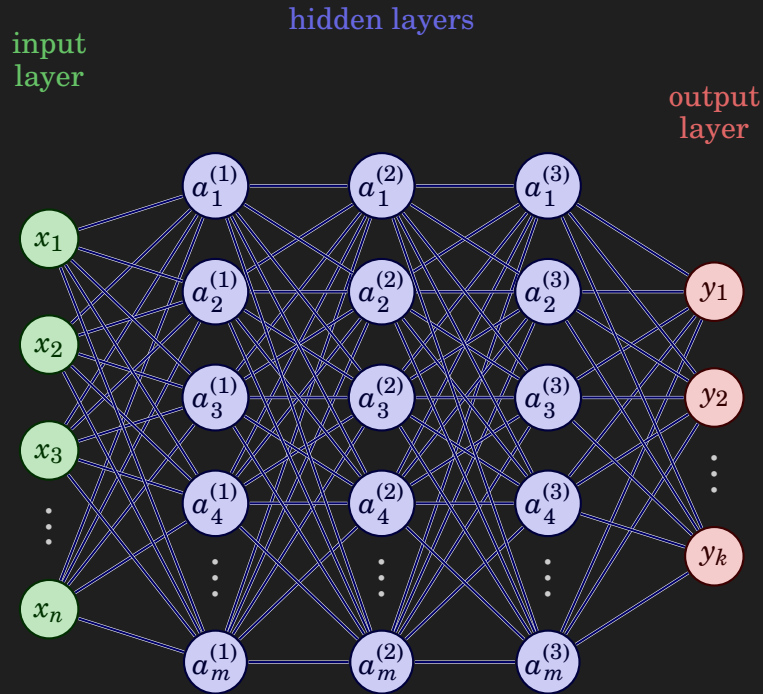
$$\frac{\partial \Delta}{\partial w_{11}^{(1)}} = \delta_1^{(2)} \cdot \left(\frac{\partial O_{h1}}{\partial w_{11}^{(1)}} \right) = \delta_1^{(2)} \cdot x_1$$

So, now just like the previous layer, we can get the gradient matrix for the weights from input layer to hidden layer as

$$\frac{\partial \Delta}{\partial w_{ij}^{(1)}} = \begin{bmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \end{bmatrix} \cdot \begin{bmatrix} x_1 & x_2 \end{bmatrix} = \begin{bmatrix} \delta_1^{(2)} \cdot x_1 & \delta_1^{(2)} \cdot x_2 \\ \delta_2^{(2)} \cdot x_1 & \delta_2^{(2)} \cdot x_2 \end{bmatrix}$$

The process you see is generalizable to n-dimensions. Like, the same process repeats for subsequent hidden layers.

n-Dimensional Neural Network.



Forward Propagation.

Here, we have to use vectors to simplify the equations. So, the input for the first neuron in the first hidden layer will be,

$$a_1^{(1)} = \sigma \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} \cdot \begin{bmatrix} w_{11}^{(1)} \\ w_{12}^{(1)} \\ w_{13}^{(1)} \\ \vdots \\ w_n^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \\ \vdots \\ b_n^{(1)} \end{bmatrix} \right)$$

Let's denote the vectors as a single letter (capital case) for simplicity.

$$A^{(1)} = \sigma (\mathbf{X} \cdot \mathbf{W}^{(1)} + \mathbf{B}^{(1)})$$

It repeats for the subsequent layers. So, for the output layer,

$$\mathbf{Y} = \sigma (\mathbf{A}^{(3)} \cdot \mathbf{W}^{(4)} + \mathbf{B}^{(4)})$$

Gradient Descent & Backpropagation.

Now, the Loss vector is

$$\mathbf{L} = \frac{1}{n} (\mathbf{Y} - \hat{\mathbf{Y}})^2$$

We can write the loss function as,

$$\mathcal{L} = \sum_{i=1}^n L_i, \quad \text{where } L_i \in \mathbf{L}$$

Now, how does the backpropagation work here? There are so many weights and biases, how do we calculate the gradients for each of them? Look at what we did in the 2-D case. We defined a loss term for each neuron. We can do the same here. For the output layer,

$$\delta_i^{(4)} = \frac{\partial \mathcal{L}}{\partial y_i} \cdot \frac{\partial y_i}{\partial O_{y_i}}$$

And so, we can write the loss term vector and previous layer output vector as the following and get the gradient matrix for the weights from hidden layer 3 to output layer as,

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(4)}} = \begin{bmatrix} \delta_1^{(4)} \\ \delta_2^{(4)} \\ \vdots \\ \delta_n^{(4)} \end{bmatrix} \cdot \begin{bmatrix} a_1^{(3)} & a_2^{(3)} & \dots & a_n^{(3)} \end{bmatrix} = \begin{bmatrix} \delta_1^{(4)} \cdot a_1^{(3)} & \delta_1^{(4)} \cdot a_2^{(3)} & \dots & \delta_1^{(4)} \cdot a_n^{(3)} \\ \delta_2^{(4)} \cdot a_1^{(3)} & \delta_2^{(4)} \cdot a_2^{(3)} & \dots & \delta_2^{(4)} \cdot a_n^{(3)} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_n^{(4)} \cdot a_1^{(3)} & \delta_n^{(4)} \cdot a_2^{(3)} & \dots & \delta_n^{(4)} \cdot a_n^{(3)} \end{bmatrix}$$

The first row gives us the gradients of the weights going into the first neuron of the output layer. The second row gives us the gradients of the weights going into the second neuron of the output layer and so on. This can be generalized to any hidden layer as,

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(L)}} = \begin{bmatrix} \delta_1^{(L)} \\ \delta_2^{(L)} \\ \vdots \\ \delta_n^{(L)} \end{bmatrix} \cdot \begin{bmatrix} a_1^{(L-1)} & a_2^{(L-1)} & \dots & a_n^{(L-1)} \end{bmatrix} = \begin{bmatrix} \delta_1^{(L)} \cdot a_1^{(L-1)} & \delta_1^{(L)} \cdot a_2^{(L-1)} & \dots & \delta_1^{(L)} \cdot a_n^{(L-1)} \\ \delta_2^{(L)} \cdot a_1^{(L-1)} & \delta_2^{(L)} \cdot a_2^{(L-1)} & \dots & \delta_2^{(L)} \cdot a_n^{(L-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_n^{(L)} \cdot a_1^{(L-1)} & \delta_n^{(L)} \cdot a_2^{(L-1)} & \dots & \delta_n^{(L)} \cdot a_n^{(L-1)} \end{bmatrix}$$

Where,

$$\delta_j^{(L)} = \mathbf{A}' \left(a_j^{(L)} \right) \cdot \sum_{i=1}^n \delta_i^{(L+1)} \cdot w_{ij}^{(L)}$$

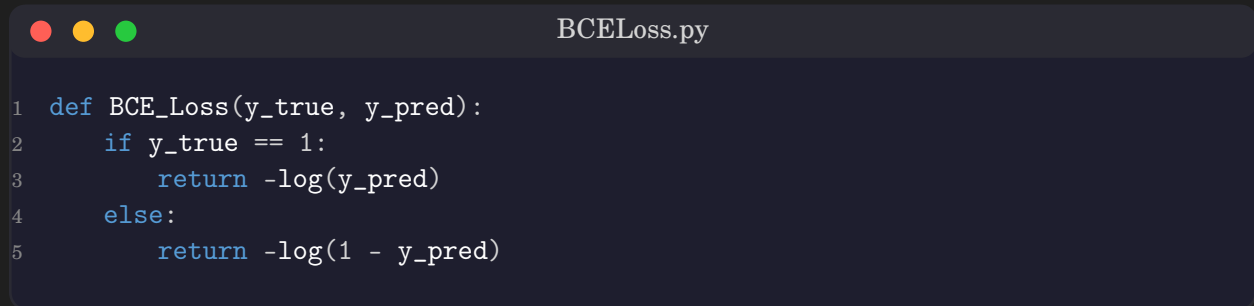
Loss Functions.

Binary Cross Entropy Loss.

Useful for binary classification problems. It measures the difference between two probability distributions - the true labels and the predicted probabilities. It is given by,

$$\text{BCE Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

This is how we can implement it in python.



```
1 def BCE_Loss(y_true, y_pred):
2     if y_true == 1:
3         return -log(y_pred)
4     else:
5         return -log(1 - y_pred)
```

Convolutional Neural Networks.